3D 게임프로그래밍 과제 2

제출일	2024. 00. 00	**전공**	게임공학과
과목	3D 게임프로그래밍	**학번**	2019182025
담당교수	이용희	**이름**	원정우

목차

1. 과제에 대한 목표 3

2. 가정 3

3. 실행결과 3

3.1 START 실행결과 장면 3

3.2 Stage 실행화면 4

4. 조작법 4

4.1 Start Scene 4

4.2 Stage Scene 5

5. 코드 6

5.1 씬 전환 6

5.2 플레이어 – 회전, 카메라 10

5.3 플레이어 – 총, 폭발, 실드 11

5.4 총알, 적 비행기 15

5.5 충돌처리 18

1. 과제에 대한 목표

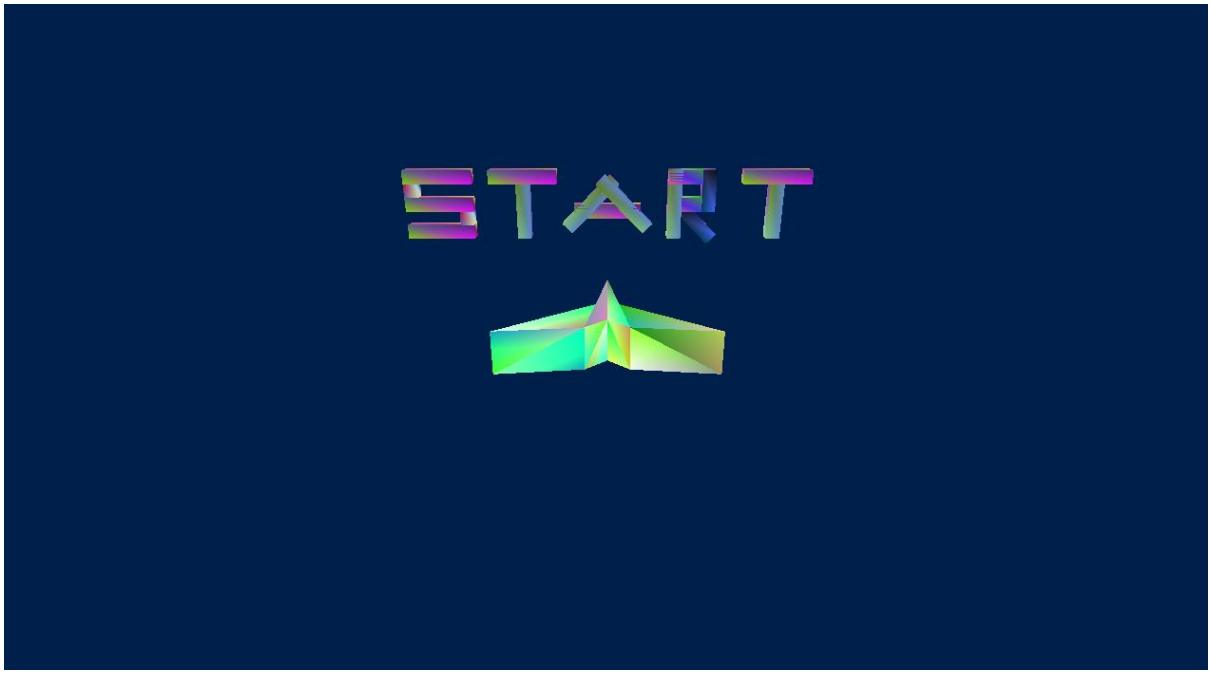
과제 1에서 제출했던 방식을 그대로 구현하고 수업에서 배운 부분을 최대한 활용한다. 과제1에서처럼 플레이어가 벽에 부딪히면 벽을 이동시키는 것이 아닌 반사벡터를 구해 적 비행기처럼 튕기게 구현해본다. 쉐이더를 여러 개를 이용해서 화면을 구성하게 해본다. D3D 12에 대한 이해도를 높이는 방식을 최대한 활용한다.

2. 가정

물체를 선택할 수 있어야 하기 때문에 픽킹의 기능 구현이 필요하다. 그렇기 때문에 따라하기 17번까지 따라하며 만든 프로그램을 이용하여 만든다. 기존 코드에 이미 충돌, 카메라, 플레이어, 오브젝트가 어느정도 구현 되어있기 때문에 그 부분에 추가하며 만들면 될 것 같다.

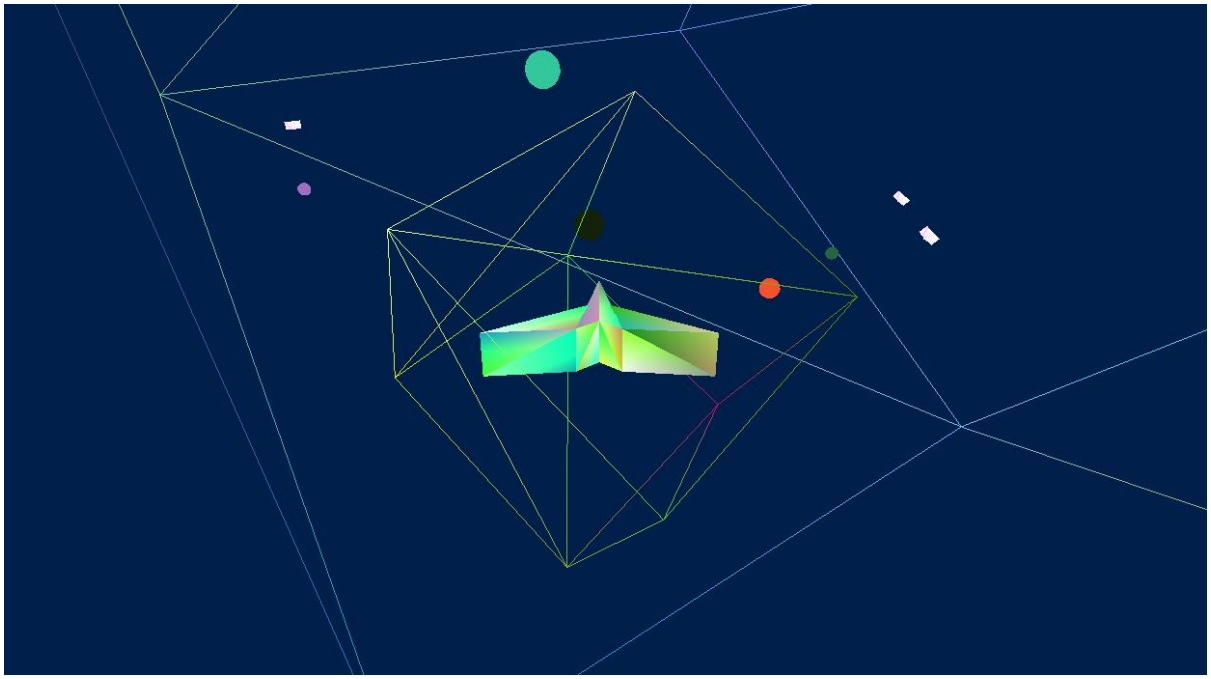
3. 실행결과

3.1 START 실행결과 장면



과제 1과는 다르게 면으로 채워 START를 만들었고 START의 글자 중 어느 것을 눌러도 stage scene으로 넘어가게 만들었다.

3.2 Stage 실행화면



과제 1처럼 적 비행기가 이동하며 플레이어를 향해 총알을 발사한다. 과제 1과 다르게 총알은 육면체 대신 구체를 이용했다. 또한 벽을 채워서 사용하려고 했으나 시야적으로 불편함이 있어 와이어 프레임으로 구성했다.

4. 조작법

4.1 Start Scene

키보드는 F1 - 1인칭 화면 / F2 – 스페이스쉽 화면 / F3 – 3인칭 화면 / control 키 – 총알 발사이며 각 화살표로 비행기 기준 앞뒤 좌우를 이동할 수 있으며 page업, page다운 키를 통해 비행기 기준 위, 아래를 이동할 수 있다.

마우스는 start를 클릭하면 다음 stage scene으로 넘어가게 되며 아닌 부분을 누르고 드래그를 하면 과제 1번처럼 화면을 움직일 수 있다. 1인칭과 3인칭인 경우에도 화면을 자유롭게 움직일 수 있게 만들어 두었고 방향에 따라 비행기도 움직이도록 구현했다.

마우스와 방향키만으로 자유롭게 움직일 수 있게 구현했다.

4.2 Stage Scene

기존 Start Scene에 있던 조작을 기본적으로 가지고 있다. 추가적으로 1~7번 키를 누르면 순서에 맞는 비행기가 폭발하게 된다.

```
        break;
case '7': {
    CExplosiveObject* pExplosiveObject = (CExplosiveObject*)m_pShaders[0]->m_ppObjects[int(wParam - '1')];
    pExplosiveObject->m_bBlowingUp = true;
}
        break;

case 'A':
    for (int i = 0; i < m_pShaders[0]->m_nObjects; i++)
    {
        CExplosiveObject* pExplosiveObject = (CExplosiveObject*)m_pShaders[0]->m_ppObjects[i];
        pExplosiveObject->m_bBlowingUp = true;
    }
    break;
case 'X':
    m_pPlayer->m_shield = false;
    break;
case 'Z':
    m_pPlayer->m_shield = true;
    break;
```

z로 플레이어 비행기의 실드를 키고 x로 플레이어 비행기의 실드를 끌 수 있다. 추가적으로 a를 누르면 모든 적 비행기가 폭발하게 된다. 마우스로 적 비행기를 클릭하면 그 비행기를 선택하게 되고(색이 바뀌지는 않는다.) 그 이후 cotrol키를 눌러 총을 쏘면 선택된 적 비행기를 추격하는 총알을 발사한다.

[1]

[1] 전체적인 코드는 길고 큰 의미가 있지 않기 때문에 기능이 다른 부분만 첨부하였습니다.

5. 코드

5.1 씬 전환

```
class start_Scene :CScene
{
public:
    start_Scene();
    ~start_Scene();


    virtual void BuildObjects(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList *pd3dCommandList);

};

class stage_Scene :CScene
{
public:

    stage_Scene();
    ~stage_Scene();

    virtual void stage_Scene::AnimateObjects(float fTimeElapsed);
    virtual bool OnProcessingKeyboardMessage(HWND hWnd, UINT nMessageID, WPARAM wParam, LPARAM
        lParam);

    virtual void CheckPlayerByWallCollisions();
    virtual void CheckObjectByWallCollisions();

    virtual void BuildObjects(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList *pd3dCommandList);
};
```

CScene은 기존 코드에서 충돌처리를 하는 함수만 추가했다. 각 스테이지를 구현하기 위해서 2가지 클래스를 추가했고 state_Scene에선 벽과의 충돌을 위해 2개의 함수[2]를 구현했다.

```
//씬은 게임 객체들의 집합이다. 게임 객체는 셰이더를 포함한다.
ID3D12RootSignature* m_pd3dGraphicsRootSignature = NULL;
CObjectsShader** m_pShaders = NULL;
int m_nShaders = 0;
CPlayer* m_pPlayer = NULL;
```

또한 쉐이더를 여러 개를 이용하기 위해 포인터의 배열로 수정했다. 쉐이더는 CstartShader, CstageShader, CwallShader가 있으며 CstartShader는 start 씬에서 CstageShader, CwallShader는 stage 씬에서 사용된다.

[2] 벽과 충돌 함수는 이후에 충돌설명 부분에 있다.

```
type = 0;
CExplosiveObject::PrepareExplosion(pd3dDevice, pd3dCommandList);

m_pd3dGraphicsRootSignature = CreateGraphicsRootSignature(pd3dDevice);
m_nShaders = 1;
m_pShaders = new CObjectsShader * [m_nShaders];

CstartShader* startShader = new CstartShader();
m_pShaders[0] = startShader;
m_pShaders[0]->CreateShader(pd3dDevice, m_pd3dGraphicsRootSignature);
m_pShaders[0]->BuildObjects(pd3dDevice, pd3dCommandList);
```

start 신에서는 CstartShader를 사용하며

```
void CstartShader::BuildObjects(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList)
{
    CCubeMeshDiffused* pCubeMeshRight = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList,20.0f, 4.0f, 4.0f);
    CCubeMeshDiffused* pCubeMeshUp = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList, 4.0f, 20.0f, 4.0f);
    CCubeMeshDiffused* pCubeMesh = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList, 4.0f, 4.0f, 4.0f);
    CCubeMeshDiffused* pCubeMeshShort = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList, 10.0f, 4.0f, 4.0f);
    CCubeMeshDiffused* pCubeMeshShortUp = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList, 4.0f, 12.0f, 4.0f);

    m_nObjects = 17;
    m_ppObjects = new CGameObject * [m_nObjects];

    CGameObject* pRotatingObject = NULL;
    CGameObject* pExplosiveObject = new CGameObject();
    pExplosiveObject->SetMesh(pCubeMeshRight);
    pExplosiveObject->SetPosition(-50.0f, 0.0f, 80.0f);
    pExplosiveObject->SetMovingDirection(XMFLOAT3(0.0f, 0.0f, 0.0f));
    pExplosiveObject->SetMovingSpeed(0.0f);
    m_ppObjects[0] = pExplosiveObject;
```

CstartShader의 BuildObjects에서 과제 1과 같이 큐브 메쉬를 이용한 START의 도형배치가 이뤄진다.

```
m_pd3dGraphicsRootSignature = CreateGraphicsRootSignature(pd3dDevice);
m_nShaders = 2;
m_pShaders = new CObjectsShader * [m_nShaders];

CstageShader* stageShader = new CstageShader;
m_pShaders[0] = (CObjectsShader*)stageShader;
m_pShaders[0]->CreateShader(pd3dDevice, m_pd3dGraphicsRootSignature);
m_pShaders[0]->BuildObjects(pd3dDevice, pd3dCommandList);


CwallShader* wallShader = new CwallShader;
m_pShaders[1] = (CObjectsShader*)wallShader;
m_pShaders[1]->CreateShader(pd3dDevice, m_pd3dGraphicsRootSignature);
m_pShaders[1]->BuildObjects(pd3dDevice, pd3dCommandList);
```

stage 신에서는 CsageShader와 CwallShader를 2개를 이용한다. 여기서 m_pShaders[0]에는 적 비행기 오브젝트들을 m_pShaders[1]에서는 벽을 넣기로 정했다.

```
CAirplaneMeshDiffused* pairplane = new CAirplaneMeshDiffused(pd3dDevice, pd3dCommandList,
    8.0f, 12.0f, 4.0f, RANDOM_COLOR);
m_nObjects = 7;

m_ppObjects = new CGameObject * [m_nObjects];
CExplosiveObject* pRotatingObject = NULL;

for (int i = 0; i < 7; ++i) {
    pRotatingObject = new CExplosiveObject(pd3dDevice, pd3dCommandList);
    pRotatingObject->SetMesh((CMesh*)pairplane);
    pRotatingObject->SetPosition(float(uidPos(dre)), float(uidPos(dre)), float(uidPos(dre)));
    pRotatingObject->SetRotationAxis(XMFLOAT3(1.0f, 0.0f, 0.0f));
    pRotatingObject->SetRotationSpeed(0);
    pRotatingObject->type = 'A';
    pRotatingObject->SetMovingDirection(XMFLOAT3(float(uidDir(dre)), float(uidDir(dre)), float(uidDir(dre))));
    pRotatingObject->SetMovingSpeed(30.0f);

    m_ppObjects[i] = pRotatingObject;
}
CreateShaderVariables(pd3dDevice, pd3dCommandList);
```

먼저 CstageShader의 BuildObjects에서는 비행기 메쉬를 가져오며 총 7개의 적 비행기를 생성하며 랜덤한 위치와 랜덤한 이동 방향을 정해준다.

```
D3D12_RASTERIZER_DESC CwallShader::CreateRasterizerState()
{
    D3D12_RASTERIZER_DESC d3dRasterizerDesc;
    ::ZeroMemory(&d3dRasterizerDesc, sizeof(D3D12_RASTERIZER_DESC));
    d3dRasterizerDesc.FillMode = D3D12_FILL_MODE_WIREFRAME;
    d3dRasterizerDesc.CullMode = D3D12_CULL_MODE_NONE;
```

CwallShader에서는 면을 채우지 않기 위해 D3D12_FILL_MODE_WIREFRAME을 사용하였고 벽 오브젝트 내부에서 이동하기 때문에 D3D12_CULL_MODE_NONE을 통해 배경이 은면처리 되지 않게 했다.

```
void CwallShader::BuildObjects(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList)
{

    CCubeMeshDiffused* wallplane = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList,
        400.0f, 400.0f, 400.0f);

    m_nObjects = 1;
    m_ppObjects = new CGameObject * [m_nObjects];

    CWallObject* pCWallObject = NULL;
    pCWallObject = new CWallObject(pd3dDevice, pd3dCommandList);
    pCWallObject->SetMesh((CMesh*)wallplane);
    pCWallObject->SetPosition(0.0f, 0, 0);

    m_ppObjects[0] = pCWallObject;

    CreateShaderVariables(pd3dDevice, pd3dCommandList);
```

CwallShader의 BuildObjects에서는 큐브 메쉬를 벽으로 정했고 벽은 클릭되지 않게 CwallShader::PickObjectByRayIntersection를 추가하고 해당 함수에서는 무조건 NULL을 리턴하게

해 두었다.

```
case WM_RBUTTONDOWN:
    //마우스 캡처를 하고 현재 마우스 위치를 가져온다.
    m_pSelectedObject = m_pScene->PickObjectPointedByCursor(LOWORD(lParam),
        HIWORD(lParam), m_pCamera);
    if (m_pSelectedObject && m_pScene->type == 0)
    {
        BuildObjects_change();
    }
    ::SetCapture(hWnd);
    ::GetCursorPos(&m_ptOldCursorPos);
```

GameFramework에서 만약 start를 이루는 오브젝트들 중 하나라도 클릭이 되고 start씬이라면 BuildObjects_change 함수가 실행이 된다.

```
void CGameFramework::BuildObjects_change()
{
    ReleaseObjects();
    m_pd3dCommandList->Reset(m_pd3dCommandAllocator, NULL);

    stage_Scene* check_scene = new stage_Scene();
    m_pScene = (CScene*)check_scene;
    if (m_pScene) m_pScene->BuildObjects(m_pd3dDevice, m_pd3dCommandList);

    CAirplanePlayer* pAirplanePlayer = new CAirplanePlayer(m_pd3dDevice,
        m_pd3dCommandList, m_pScene->GetGraphicsRootSignature());
    m_pPlayer = pAirplanePlayer;
    m_pCamera = m_pPlayer->GetCamera();

    m_pd3dCommandList->Close();

    m_pScene->setPlayer(m_pPlayer);
    ID3D12CommandList* ppd3dCommandLists[] = { m_pd3dCommandList };
    m_pd3dCommandQueue->ExecuteCommandLists(1, ppd3dCommandLists);

    WaitForGpuComplete();
    if (m_pScene) m_pScene->ReleaseUploadBuffers();
    m_GameTimer.Reset();
}
```

BuildObjects_change 함수에서는 먼저 기존에 있던 것들을 지운 뒤에 stage_Scene으로 다시 생성하는 방식이다.

5.2 플레이어 – 회전, 카메라

```
else if ((nCameraMode == THIRD_PERSON_CAMERA))
{
    m_pCamera->Rotate(x, y, z);
    if (x != 0.0f)
    {
        XMMATRIX xmmtxRotate = XMMatrixRotationAxis(XMLoadFloat3(&m_xmf3Right),
            XMConvertToRadians(x));
        m_xmf3Look = Vector3::TransformNormal(m_xmf3Look, xmmtxRotate);
        m_xmf3Up = Vector3::TransformNormal(m_xmf3Up, xmmtxRotate);
    }
    if (y != 0.0f)
    {
        XMMATRIX xmmtxRotate = XMMatrixRotationAxis(XMLoadFloat3(&m_xmf3Up),
            XMConvertToRadians(y));
        m_xmf3Look = Vector3::TransformNormal(m_xmf3Look, xmmtxRotate);
        m_xmf3Right = Vector3::TransformNormal(m_xmf3Right, xmmtxRotate);
    }
    if (z != 0.0f)
    {
        XMMATRIX xmmtxRotate = XMMatrixRotationAxis(XMLoadFloat3(&m_xmf3Look),
            XMConvertToRadians(z));
        m_xmf3Up = Vector3::TransformNormal(m_xmf3Up, xmmtxRotate);
        m_xmf3Right = Vector3::TransformNormal(m_xmf3Right, xmmtxRotate);
    }
}
```

```
CFirstPersonCamera::CFirstPersonCamera(CCamera* pCamera) : CCamera(pCamera)
{
    m_nMode = FIRST_PERSON_CAMERA;
    if (pCamera)
    {
        if (pCamera->GetMode() == SPACESHIP_CAMERA || pCamera->GetMode() == THIRD_PERSON_CAMERA)
        {
```

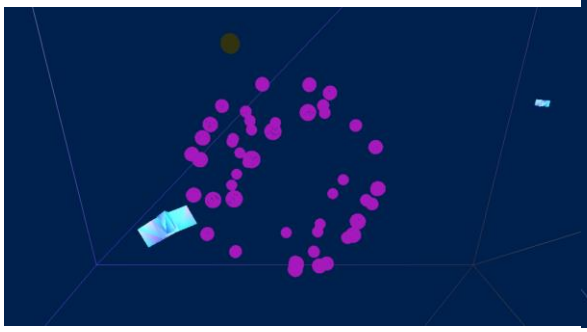
과제 1번처럼 1인칭은 그대로 두고 3인칭을 구현하기 위해 따로 분리해서 스페이스 – 쉽과 동일하게 만들었다. 마우스 클릭 후 움직이면 오른쪽 그림과 같이 원하는 대로 회전시킬 수 있다. 밑의 사진과 같이 3인칭에서도 원하는 곳으로 화면을 움직이고 총을 쏠 수 있다.

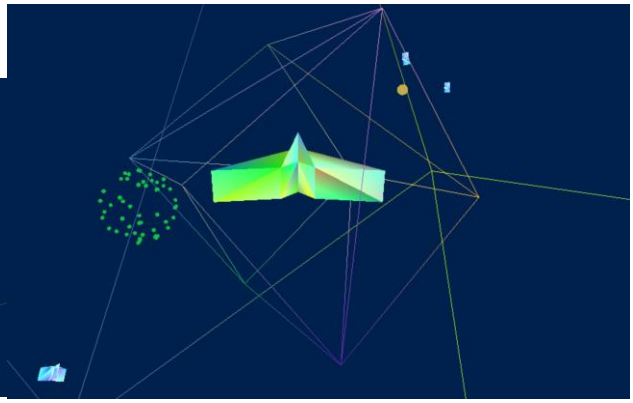
5.3 플레이어 – 총, 폭발, 실드

```
// "CPlayer" 클래스를 다음과 같이 수정한다.
class CPlayer : public CGameObject
{
public:
    //터지기
    bool                    m_bBlowingUp = false;
    static CMesh* m_pExplosionMesh;
    static XMFLOAT3         m_pxmf3SphereVectors[EXPLOSION_DEBRISES];
    XMFLOAT4X4              m_pxmf4x4Transforms[EXPLOSION_DEBRISES];


    float                   m_fElapsedTimes = 0.0f;
    float                   m_fDuration = 2.0f;
    float                   m_fExplosionSpeed = 10.0f;
    float                   m_fExplosionRotation = 720.0f;

    bool                    m_shield = false;
    CGameObject* m_pshield;

public:
    virtual void Animate(float fElapsedTime);
```

```
class CAirplanePlayer : public CPlayer
{
public:
  //터지는 함수
  void PrepareExplosion(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList);
  //총알
  float                     m_fBulletEffectiveRange = 150.0f;
  CBulletObject*            m_ppBullets[BULLETS];

  void FireBullet(CGameObject* pLockedObject);
  // 수정된 부분
  virtual void Animate(float fElapsedTime);
  virtual void Render(ID3D12GraphicsCommandList* pd3dCommandList, CCamera* pCamera);
  //실드
  void SetShield(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList);
```

먼저 과제 1에서 사용했던 것처럼 터질 때 사용할 이펙트와 총알 발사, 실드를 만들기 위해 CPlayer와 CAirplane player에 필요한 변수와 함수를 추가하였다.





```
CAirplanePlayer::CAirplanePlayer(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList
    * pd3dCommandList, ID3D12RootSignature* pd3dGraphicsRootSignature)
{

    CSphereMeshDiffused* pBulletMesh = new CSphereMeshDiffused(pd3dDevice, pd3dCommandList,
        2.0f, 20, 20);
    for (int i = 0; i < BULLETS; i++)
    {
        m_ppBullets[i] = new CBulletObject(m_fBulletEffectiveRange);
        m_ppBullets[i]->SetMesh(pBulletMesh);
        m_ppBullets[i]->SetRotationAxis(XMFLOAT3(0.0f, 1.0f, 0.0f));
        m_ppBullets[i]->SetRotationSpeed(40.0f);
        m_ppBullets[i]->SetMovingSpeed(120.0f);
        m_ppBullets[i]->SetActive(false);
    }
    //플레이어(비행기) 메쉬를 렌더링할 때 사용할 셰이더를 생성한다.
    CPlayerShader* pShader = new CPlayerShader();
    pShader->CreateShader(pd3dDevice, pd3dGraphicsRootSignature);
    SetShader(pShader);

    PrepareExplosion(pd3dDevice, pd3dCommandList);
    SetShield(pd3dDevice, pd3dCommandList);
```

CAirplanePlayer 생성자에서 총알을 생성하고 폭발과 실드를 생성하는 함수를 호출한다. 총알의 수는 10개이며 총알의 수만큼 배열에 생성한다.

```
void CAirplanePlayer::PrepareExplosion(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList
    * pd3dCommandList)
{
    for (int i = 0; i < EXPLOSION_DEBRISES; i++) XMStoreFloat3(&m_pxmf3SphereVectors[i], ::RandomUnitVectorOnSphere2());
    m_pExplosionMesh = new CSphereMeshDiffused(pd3dDevice, pd3dCommandList, 1.0f, 20, 20);
}

void CAirplanePlayer::SetShield(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList
    * pd3dCommandList)
{

    CCubeMeshDiffused* pCubeMesh = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList, 25.0f, 25.0f, 25.0f);

    CRotatingObject* pExplosiveObject = new CRotatingObject();
    pExplosiveObject->SetMesh(pCubeMesh);
    pExplosiveObject->SetPosition(GetPosition());
    pExplosiveObject->SetRotationAxis(XMFLOAT3(0.0f, 1.0f, 0.0f));
    pExplosiveObject->SetRotationSpeed(50.0f);
    pExplosiveObject->SetMovingDirection(XMFLOAT3(0.0f, 0.0f, 0.0f));
    pExplosiveObject->SetMovingSpeed(0.0f);

    m_pshield = pExplosiveObject;
}
```

폭발은 과제 1과 동일하며 실드는 오브젝트를 하나 생성한 뒤에 m_pshield에 넣어준다. 실드의 메쉬는 큐브를 사용했다. PrepareExplosion에서 사용되는 RandomUnitVectorOnSphere2과 그 함수에서 사용되는 RandF2는 과제 1의 RandomUnitVectorOnSphere와 RandF 함수와 동일하다.

```
if (m_bBlowingUp)
{
    m_fElapsedTimes += fElapsedTime;
    if (m_fElapsedTimes <= m_fDuration)
    {
        XMFLOAT3 xmf3Position = GetPosition();
        for (int i = 0; i < EXPLOSION_DEBRISES; i++)
        {
            m_pxmf4x4Transforms[i] = Matrix4x4::Identity();
            m_pxmf4x4Transforms[i]._41 = xmf3Position.x + m_pxmf3SphereVectors[i].x * m_fExplosionSpeed * m_fElapsedTimes;
            m_pxmf4x4Transforms[i]._42 = xmf3Position.y + m_pxmf3SphereVectors[i].y * m_fExplosionSpeed * m_fElapsedTimes;
            m_pxmf4x4Transforms[i]._43 = xmf3Position.z + m_pxmf3SphereVectors[i].z * m_fExplosionSpeed * m_fElapsedTimes;
            m_pxmf4x4Transforms[i] = Matrix4x4::Multiply(Matrix4x4::RotationAxis(m_pxmf3SphereVectors[i], m_fExplosionRotation * m_fElapsedTimes), m_pxmf4x4Transforms[i]);
        }
    }
    else
    {


        m_fElapsedTimes = 0;
        m_bBlowingUp = false;
```

먼저 터지는 부분을 설명하면 충돌처리 함수에서 만약 플레이어가 충돌했다는 것을 확인하면 과제 1과 동일한 방식으로 진행된다. m_bBlowingUp 변수가 true로 바뀌게 되고 animation에서 맞은 위치에서 터지는 애니메이션을 진행하고 Render에서 터지는 이펙트가 그려지게 된다

```
void CPlayer::Render(ID3D12GraphicsCommandList* pd3dCommandList, CCamera* pCamera)
{
    DWORD nCameraMode = (pCamera) ? pCamera->GetMode() : 0x00;

    if (m_bBlowingUp)
    {
        for (int i = 0; i < EXPLOSION_DEBRISES; i++)
        {
            CGameObject::Render(pd3dCommandList, pCamera, &m_pxmf4x4Transforms[i], m_pExplosionMesh);
        }
    }
    else
    {
        if (m_shield == true)
        {
            m_pshield->Render(pd3dCommandList, pCamera);
        }
        if (m_pShader)
        {
            m_pShader->Render(pd3dCommandList, pCamera);
        }
        if (nCameraMode == THIRD_PERSON_CAMERA)
        {
            CGameObject::Render(pd3dCommandList, pCamera);
        }
    }

}
```

실드 또한 Z키를 눌러 실드가 켜지게 되면 m_shield의 값이 true로 바뀌게 되며 회전하는 육면체의 실드가 보이게 되며 충돌을 실드가 막아준다. Render에서는 만약 3인칭이 아닌 경우에도 실드와 터지는 이펙트는 보이게 만들었다.

```
void CGameObject::Render(ID3D12GraphicsCommandList* pd3dCommandList, CCamera* pCamera, XMFLOAT4X4* pxmf4x4World, CMesh* pMesh)
{
    XMFLOAT4X4 xmmtx4x4TransposedWorld = Matrix4x4::Transpose(*pxmf4x4World);

    pd3dCommandList->SetGraphicsRoot32BitConstants(0, 16, &xmmtx4x4TransposedWorld, 0);

    if (pMesh)
    {
        pMesh->Render(pd3dCommandList);
    }
}
```

터지는 이펙트는 SetGraphicsRoot32BitConstants를 통해 Render가 진행된다. 여기서 열 우선으로 들어온 pxmf4x4World행렬을 행 우선 행렬로 바꿔서 사용하기 위해 Transpose를 해준 뒤 사용했다.

```
void CAirplanePlayer::FireBullet(CGameObject* pLockedObject)
{
    CBulletObject* pBulletObject = NULL;
    for (int i = 0; i < BULLETS; i++)
    {
        if (!m_ppBullets[i]->m_bActive)
        {
            pBulletObject = m_ppBullets[i];
            break;
        }
    }

    if (pBulletObject)
    {
        XMFLOAT3 xmf3Position = GetPosition();
        XMFLOAT3 xmf3Direction = GetUp();
        XMFLOAT3 xmf3FirePosition = Vector3::Add(xmf3Position, Vector3::ScalarProduct(xmf3Direction, 6.0f, false));

        pBulletObject->m_xmf4x4World = m_xmf4x4World;

        pBulletObject->SetFirePosition(xmf3FirePosition);
        pBulletObject->SetMovingDirection(xmf3Direction);
        pBulletObject->SetActive(true);

        if (pLockedObject)
        {
            pBulletObject->m_pLockedObject = pLockedObject;
        }
    }
}
```

총알은 control 키를 누르면 한발을 발사하고 플레이어 비행기에서 발사되며 방향은 GetUp을 통해 가져온다. 만약 선택된 적 비행기가 있다면 pLockedobject에 저장되며 그 비행기를 따라가는 총알을 발사한다.

5.4 총알, 적 비행기

```
class CGameObject
{
public:
    char type{'N'};
    XMFLOAT3                m_xmf3MovingDirection = XMFLOAT3(0.0f, 0.0f, 1.0f);
    float                   m_fMovingSpeed = 0.0f;
    float                   m_fMovingRange = 0.0f;

    void SetMovingDirection(XMFLOAT3& xmf3MovingDirection) { m_xmf3MovingDirection = Vector3::Normalize(xmf3MovingDirection); }
    void SetMovingSpeed(float fSpeed) { m_fMovingSpeed = fSpeed; }
    void SetMovingRange(float fRange) { m_fMovingRange = fRange; }

    bool                    m_bActive = true;

    void SetActive(bool bActive) { m_bActive = bActive; }

    XMFLOAT4X4 m_xmf4x4World;

    void Move(XMFLOAT3& vDirection, float fSpeed);


    virtual void UpdateBoundingBox();

    BoundingOrientedBox     m_xmOOBB = BoundingOrientedBox();
    //BoundingSphere            m_xmOOSS = BoundingSphere();
```

먼저 이동과 방향 등을 설정할 함수와 변수를 만들었고 총알이 활성화 되어있는지 아닌지를 확인하기 위해 m_bActive를 통해 설정한다. 만약 발사된 총알이면 true 아니라면 false의 값을 가진다. 충돌처리를 위해 위치를 계속 설정해 줄 UpdateBoundingBox 함수도 구현했다.

```
class CExplosiveObject : public CRotatingObject
{
```

```
class CBulletObject : public CRotatingObject
{
```

추가적으로 총알 오브젝트 클래스와 ExplosiveObject 클래스를 추가했다.

```
public:
    virtual void Animate(float fElapsedTime);

    float                   m_fBulletEffectiveRange = 50.0f;
    float                   m_fMovingDistance = 0.0f;
    float                   m_fRotationAngle = 0.0f;
    XMFLOAT3                m_xmf3FirePosition = XMFLOAT3(0.0f, 0.0f, 1.0f);

    float                   m_fElapsedTimeAfterFire = 0.0f;
    float                   m_fLockingDelayTime = 0.3f;
    float                   m_fLockingTime = 4.0f;

    CGameObject* m_pLockedObject = NULL;

    void SetFirePosition(XMFLOAT3 xmf3FirePosition);
    void Reset();
```

총알 오브젝트 클래스는 과제 1과 동일하며 총알과 관련된 변수와 함수들이 들어있다.

```
    bool                        m_bBlowingUp = false;

    XMFLOAT4X4                  m_pxmf4x4Transforms[EXPLOSION_DEBRISES];

    float                       m_fElapsedTimes = 0.0f;
    float                       m_fDuration = 2.0f;
    float                       m_fExplosionSpeed = 10.0f;
    float                       m_fExplosionRotation = 720.0f;

    float                       m_fElapsedTimeAfterFire = 0.0f;
    float                       m_fLockingDelayTime = 4.0f;


    CBulletObject*              m_ppBullets[BULLETS];
    float                       m_fBulletEffectiveRange = 150.0f;
    void FireBullet(float fElapsedTime);

    virtual void Animate(float fElapsedTime);
    virtual void Render(ID3D12GraphicsCommandList* pd3dCommandList, CCamera* pCamera);

public:
    static CMesh*               m_pExplosionMesh;
    static XMFLOAT3             m_pxmf3SphereVectors[EXPLOSION_DEBRISES];

    static void PrepareExplosion(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList);
```

ExplosiveObject는 오브젝트가 터지거나 총알을 쏴야 할 때 필요한 변수와 함수들을 가지고 있다.

```
m_fElapsedTimeAfterFire += fElapsedTime;

float fDistance = m_fMovingSpeed * fElapsedTime;

if (m_fElapsedTimeAfterFire > m_fLockingDelayTime ) {

    CBulletObject* pBulletObject = NULL;
    for (int i = 0; i < BULLETS; ++i)
    {
        if (!m_ppBullets[i]->m_bActive)
        {
            pBulletObject = m_ppBullets[i];
            break;
        }
    }
```

FireBullet은 일정 시간마다 총알을 발사하는 방식이며 발사되지 않은 총알을 발사한다.

```
void CExplosiveObject::Animate(float fElapsedTime)
{
    if (m_bBlowingUp) { ... }
    else
    {
        FireBullet(fElapsedTime);
        for (int i = 0; i < BULLETS; i++)
        {
            if (m_ppBullets[i]->m_bActive) m_ppBullets[i]->Animate(fElapsedTime);
        }

        CRotatingObject::Animate(fElapsedTime);
    }
}
```

CExplosiveObject::Animate함수에서 FireBullet 함수를 호출하고 정해준 딜레이 시간에 따라 일정한 주기로 총알을 발사한다. 또한 오브젝트가 충돌로 m_bBlowingUp의 값이 true가 되면 플레이어의 비행기가 터질 때처럼 animate와 render를 수행하며 총알에 대한 관리도 수행한다. 추가적으로 CRotatingObject::Animate에서는 CGameObject::Rotate와 CGameObject::Animate를 실행한다.

```
void CGameObject::Animate(float fTimeElapsed)
{
    if (m_fMovingSpeed != 0.0f) Move(m_xmf3MovingDirection, m_fMovingSpeed * fTimeElapsed);

    UpdateBoundingBox();
    if (type == 'A') {
        TargetRotate();
    }
}
void CGameObject::UpdateBoundingBox()
{
    if (m_pMesh)
    {
        m_pMesh->m_xmOOBB.Transform(m_xmOOBB, XMLoadFloat4x4(&m_xmf4x4World));
        XMStoreFloat4(&m_xmOOBB.Orientation, XMQuaternionNormalize(XMLoadFloat4(&m_xmOOBB.Orientation)));
    }
}
```

CGameObject::Animate에서는 충돌처리를 위해 UpdateBoundingBox 함수와 만약 type == 'A'인 오브젝트, 즉 적 비행기 오브젝트라면 플레이어를 추적하는 TargetRotate 함수를 실행한다. UpdateBoundingBox 함수에서는 Animate에서 계속 호출되면서 충돌박스가 업데이트된다.

```
void CScene::AnimateObjects(float fTimeElapsed)
{
    for (int i = 0; i < m_pShaders[0]->m_nObjects; i++)
    {
        m_pShaders[0]->m_ppObjects[i]->m_pLockedObject = m_pPlayer->GetPosition();
    }
```

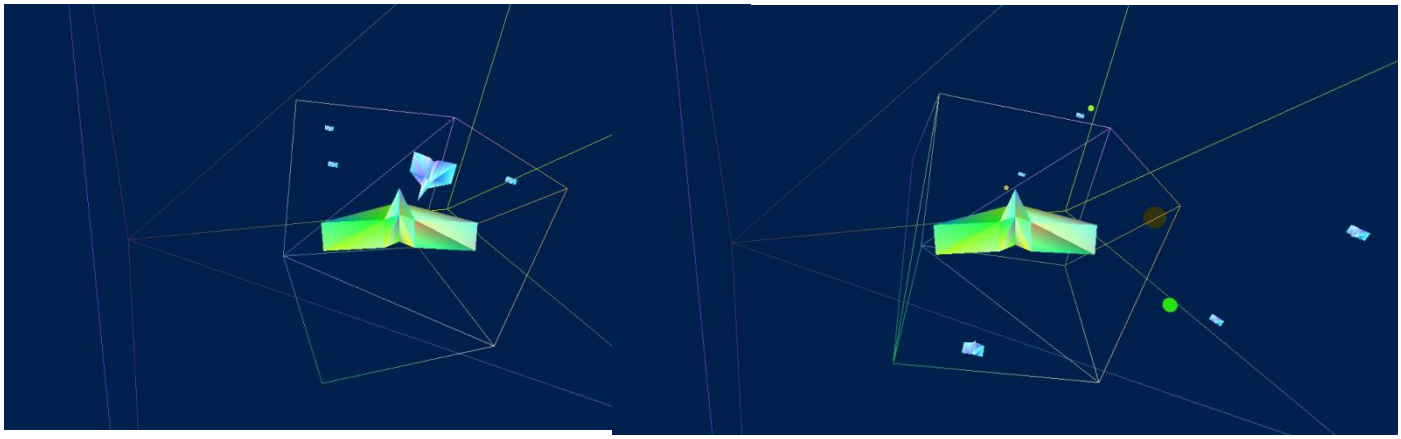
CScene에서 오브젝트들이 플레이어를 추적하기 위해 플레이어의 위치를 계속 가져온다. 그 다음 그 값을 통해 TargetRotate에서 플레이어를 바라보는 벡터를 구할 때 사용한다.

```
void CGameObject::TargetRotate()
{
    XMFLOAT3 LockedObjectPosition = m_pLockedObject;

    XMFLOAT3 xmf3Look = Vector3::Normalize(Vector3::Subtract(LockedObjectPosition, GetPosition()));
    XMFLOAT3 xmf3Right = Vector3::CrossProduct(XMFLOAT3(0.0f,1.0f,0.0f), xmf3Look, true);
    XMFLOAT3 xmf3Up = Vector3::CrossProduct(xmf3Look, xmf3Right, true);

    m_xmf4x4World._11 = xmf3Right.x; m_xmf4x4World._12 = xmf3Right.y; m_xmf4x4World._13 = xmf3Right.z;
    m_xmf4x4World._21 = xmf3Look.x; m_xmf4x4World._22 = xmf3Look.y; m_xmf4x4World._23 = xmf3Look.z;
    m_xmf4x4World._31 = xmf3Up.x; m_xmf4x4World._32 = xmf3Up.y; m_xmf4x4World._33 = xmf3Up.z;
}
```

과제 1에서는 회전각을 구해서 회전 시키는 방식을 사용했으나 이번 과제 2에서는 플레이어 비행

기를 향하는 벡터를 xmfLook을 구한 뒤에 나머지는 수직이 되도록 CrossProduct를 사용했다. 해당 함수는 CCamera::RegenerateViewMatrix() 이 함수에서 아이디어를 얻을 수 있었다. 추가적으로 비행기 오브젝트가 우리가 생각한 모습으로 쳐다보게 하기 위해 Look과 Up을 바꿔서 넣어주었다.

실행을 하면 적 비행기는 플레이어 비행기를 쳐다보며 플레이어 방향으로 총알을 발사하는 것을 볼 수 있다.



5.5 충돌처리

```
void stage_Scene::AnimateObjects(float fTimeElapsed)
{
    CScene::AnimateObjects(fTimeElapsed);

    CheckObjectByBulletCollisions();
    CheckPlyaerByBulletCollisions();
    CheckPlayerByWallCollisions();
    CheckObjectByObjectCollisions();
    CheckObjectByWallCollisions();
}
```

충돌처리는 총 5개의 함수가 있다.

```
void CScene::CheckObjectByBulletCollisions()
{
    CBulletObject** ppBullets = ((CAirplanePlayer*)m_pPlayer)->m_ppBullets;

    for (int i = 0; i < m_pShaders[0]->m_nObjects; i++)
    {
        for (int j = 0; j < BULLETS; j++)
        {
            if (ppBullets[j]->m_bActive && m_pShaders[0]->m_ppObjects[i]->m_xmOOBB.Intersects(ppBullets[j]->m_xmOOBB))
            {
                CExplosiveObject* pExplosiveObject = (CExplosiveObject*)m_pShaders[0]->m_ppObjects[i];
                pExplosiveObject->m_bBlowingUp = true;
                ppBullets[j]->Reset();
            }
        }
    }
}
```

먼저 CheckObjectByBulletCollisions 함수는 총알과 오브젝트 충돌처리이며 플레이어의 총알과 적 비행기들의 충돌이 있으면 맞은 비행기는 m_bBlowingUp가 true가 되면서 터지는 모습을 보여주게 되며 해당 총알은 Reset 함수를 통해 값을 초기화 한다.

```
void CScene::CheckPlyaerByBulletCollisions()
{
    for (int i = 0; i < m_pShaders[0]->m_nObjects; i++)
    {
        CBulletObject** ppBullets = ((CExplosiveObject*)m_pShaders[0]->m_ppObjects[i])->m_ppBullets;

        for (int j = 0; j < BULLETS; j++)
        {
            if (!m_pPlayer->m_shield) {
                if (ppBullets[j]->m_bActive && m_pPlayer->m_xmOOBB.Intersects(ppBullets[j]->m_xmOOBB))
                {
                    m_pPlayer->m_bBlowingUp = true;
                    ppBullets[j]->Reset();
                }
            }
            else
            {
                if (ppBullets[j]->m_bActive && m_pPlayer->m_pshield->m_xmOOBB.Intersects(ppBullets[j]->m_xmOOBB))
                {
                    ppBullets[j]->Reset();
                }
            }

        }
    }

}
```

CheckPlyaerByBulletCollisions 함수는 플레이어 비행기와 적 비행기들의 총알들의 충돌을 처리하며 만약 실드가 꺼져있으면 m_bBlowingUp가 true로 터지는 모습을 보여주게 되며 실드가 있는 경우에는 실드가 대신 충돌처리를 한다.

```
void CScene::CheckPlayerByWallCollisions()
{
}
void CScene::CheckObjectByWallCollisions()
{

}

void stage_Scene::CheckPlayerByWallCollisions()
{
    if (m_nShaders > 1) { ... }
}
```

벽이 있는 stage 씬에서만 벽과의 충돌처리를 하도록 해두었다. 추가적으로 첫 번째 쉐이더가 적 비행기 오브젝트, 두 번째가 벽 오브젝트를 관리하기 때문에 벽 쉐이더가 있을 때 실행하도록 조건문으로 걸어두었다.

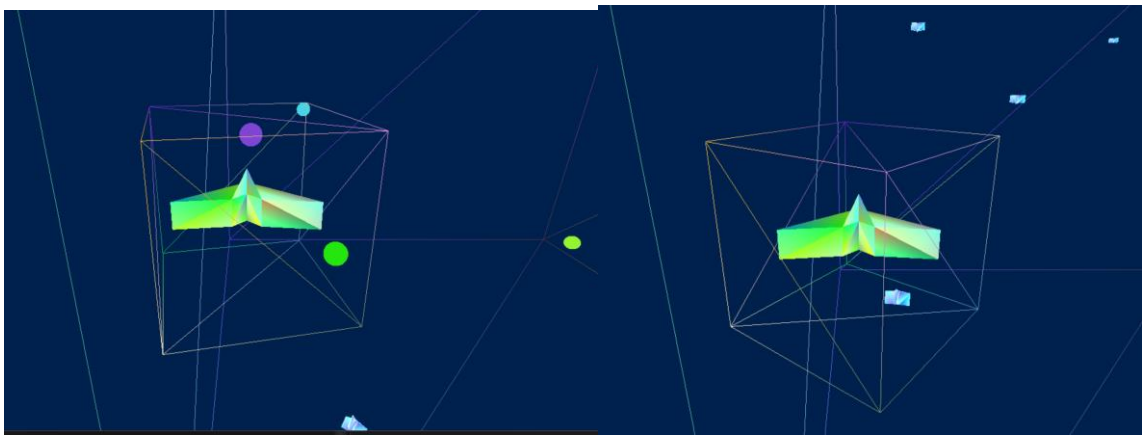
```
if (m_nShaders > 1) {
    for (int i = 0; i < m_pShaders[1]->m_nObjects; i++)
    {
        for (int j = 0; j < m_pShaders[0]->m_nObjects; j++)
        {
            ContainmentType containType = m_pShaders[1]->m_ppObjects[i]->m_xmOOBB.Contains(m_pShaders[0]->m_ppObjects[j]->m_xmOOBB);
            //ContainmentType containType = m_pShaders[1]->m_ppObjects[i]->m_xmOOSS.Contains(m_pPlayer->m_xmOOBB);
            switch (containType)
            {
            case DISJOINT:
            {
                XMFLOAT3 wallPos = m_pShaders[1]->m_ppObjects[i]->GetPosition();

                XMFLOAT3 ObjectPos = m_pShaders[0]->m_ppObjects[j]->GetPosition();

                float& x = ObjectPos.x;
                float& y = ObjectPos.y;
                float& z = ObjectPos.z;

                if (std::abs(x) >= std::abs(y) && std::abs(x) >= std::abs(z)) {
                    x = (x < 0 ? -200.0f : 200.0f); y = 0.0f; z = 0.0f;
                }
                else if (std::abs(y) >= std::abs(x) && std::abs(y) >= std::abs(z)) {
                    x = 0.0f; y = (y < 0 ? -200.0f : 200.0f); z = 0.0f;
                }
                else {
                    x = 0.0f; y = 0.0f; z = (z < 0 ? -200.0f : 200.0f);
                }

                XMFLOAT3 normalVec = Vector3::Normalize(Vector3::Subtract(wallPos, XMFLOAT3(x, y, z)));

                XMVECTOR xmvNormal = XMVectorSet(normalVec.x, normalVec.y, normalVec.z, 0.0f);
                XMVECTOR xmvReflect = XMVector3Reflect(XMLoadFloat3(&m_pShaders[0]->m_ppObjects[j]->m_xmf3MovingDirection), xmvNormal);
                XMStoreFloat3(&m_pShaders[0]->m_ppObjects[j]->m_xmf3MovingDirection, xmvReflect);

                break;
            }
            case INTERSECTS: { ... }
            case CONTAINS:
                break;
```

CheckObjectByWallCollisions 함수에서는 벽과 적 비행기들의 충돌을 관리한다. 먼저 벽 오브젝트는 큐브로 그 내부에서 있기 때문에 Contains을 통해 내부에 있는지 확인하며 DISJOINT와

INTERSECTS 내용은 동일하고 CONTAINS은 비어있다. 만약 벽 오브젝트와 충돌이 감지되었다면 부딪힌 적 비행기 오브젝트의 위치를 가져온다. 그 다음 각 좌표의 절대값들을 비교해서 가장 큰 값을 구한다. 벽은 움직이지 않고 큐브 모양이기 때문에 가장 절댓값이 큰 값과 그 값이 양수인지 음수인지에 따라 어느 면에 충돌한 건지 구할 수 있다. 충돌한 면과 원점이 축과 평행한 법선 벡터를 가지게 하기 위해 나머지 좌표는 0으로 만들고 해당 좌표만 200또는 -200을 가지게 했다. 그렇게 구한 값과 벽 오브젝트의 위치를 통해 법선 벡터를 구하고 반사벡터를 구하여 충돌한 적 비행기의 이동 방향을 수정한다. 만약 주석 처리된 부분처럼 맵이 구의 모양이고 충돌 구가 있다면 충돌이 일어난 좌표와 벽 오브젝트의 위치를 통해 법선 벡터를 구할 수 있다.

```
void stage_Scene::CheckPlayerByWallCollisions()
{
    if (m_nShaders > 1){
        for (int i = 0; i < m_pShaders[1]->m_nObjects; i++)
        {
            ContainmentType containType = m_pShaders[1]->m_ppObjects[i]->m_xmOOBB.Contains(m_pPlayer->m_xmOOBB);
            //ContainmentType containType = m_pShaders[1]->m_ppObjects[i]->m_xmOOSS.Contains(m_pPlayer->m_xmOOBB);
            switch (containType)
            {
            case DISJOINT:
            {
                XMFLOAT3 wallPos = m_pShaders[1]->m_ppObjects[i]->GetPosition();
                XMFLOAT3 playerPos = m_pPlayer->GetPosition();

                float& x = playerPos.x;
                float& y = playerPos.y;
                float& z = playerPos.z;

                // 절댓값이 가장 큰 값을 찾아 50.0f 또는 -50.0f로 변경하고 나머지는 0으로 설정
                if (std::abs(x) >= std::abs(y) && std::abs(x) >= std::abs(z)) {
                    x = (x < 0 ? -200.0f : 200.0f); y = 0.0f; z = 0.0f;
                }
                else if (std::abs(y) >= std::abs(x) && std::abs(y) >= std::abs(z)) {
                    x = 0.0f; y = (y < 0 ? -200.0f : 200.0f); z = 0.0f;
                }
                else {
                    x = 0.0f; y = 0.0f; z = (z < 0 ? -200.0f : 200.0f);
                }
                XMFLOAT3 normalVec = Vector3::Normalize(Vector3::Subtract(wallPos, XMFLOAT3(x,y,z)));
                XMFLOAT3 playerDirection = m_pPlayer->GetVelocity();
                float dotProduct = Vector3::DotProduct(playerDirection, normalVec);
                XMFLOAT3 reflectedDirection = Vector3::Subtract(playerDirection, Vector3::ScalarProduct(normalVec, 2.0f * dotProduct));
                m_pPlayer->SetVelocity(reflectedDirection);
                break;
            }
            case INTERSECTS: { ... }
            case CONTAINS:
                break;
            }
        }
    }
}
```

CheckPlayerByWallCollisions 함수도 CheckObjectByWallCollisions 함수와 비슷하게 진행되며 다른 부분은 m_pPlayer->GetVelocity()를 값을 이용해 반사 벡터 공식을 이용하여 반사벡터를 구한다. 구한 값을 m_pPlayer-SetVelocity(reflectedDirection)로 값을 수정한다. 이렇게 하면 벽에 부딪혔을 때 벽에서 튕겨나가는 방식으로 구현된다. 이 함수에서도 만약 벽이 구 모양이고 충돌 구를 가지고 있다면 부딪힌 위치와 벽의 위치를 통해 법선 벡터를 구할 수 있다.

```
void CScene::CheckObjectByObjectCollisions()
{
    for (int i = 0; i < m_pShaders[0]->m_nObjects; i++)m_pShaders[0]->m_ppObjects[i]->m_pObjectCollided = NULL;
    for (int i = 0; i < m_pShaders[0]->m_nObjects; i++)
    {
        for (int j = (i + 1); j < m_pShaders[0]->m_nObjects; j++)
        {
            if (m_pShaders[0]->m_ppObjects[i]->m_xmOOBB.Intersects(m_pShaders[0]->m_ppObjects[j]->m_xmOOBB))
            {
                m_pShaders[0]->m_ppObjects[i]->m_pObjectCollided = m_pShaders[0]->m_ppObjects[j];
                m_pShaders[0]->m_ppObjects[j]->m_pObjectCollided = m_pShaders[0]->m_ppObjects[i];
            }
        }
    }
    for (int i = 0; i < m_pShaders[0]->m_nObjects; i++)
    {
        if (m_pShaders[0]->m_ppObjects[i]->m_pObjectCollided)
        {
            XMFLOAT3 xmf3MovingDirection = m_pShaders[0]->m_ppObjects[i]->m_xmf3MovingDirection;
            float fMovingSpeed = m_pShaders[0]->m_ppObjects[i]->m_fMovingSpeed;
            m_pShaders[0]->m_ppObjects[i]->m_xmf3MovingDirection = m_pShaders[0]->m_ppObjects[i]->m_pObjectCollided->m_xmf3MovingDirection;
            m_pShaders[0]->m_ppObjects[i]->m_fMovingSpeed = m_pShaders[0]->m_ppObjects[i]->m_pObjectCollided->m_fMovingSpeed;
            m_pShaders[0]->m_ppObjects[i]->m_pObjectCollided->m_xmf3MovingDirection = xmf3MovingDirection;
            m_pShaders[0]->m_ppObjects[i]->m_pObjectCollided->m_fMovingSpeed = fMovingSpeed;
            m_pShaders[0]->m_ppObjects[i]->m_pObjectCollided->m_pObjectCollided = NULL;
            m_pShaders[0]->m_ppObjects[i]->m_pObjectCollided = NULL;
        }
    }
}
```

CheckObjectByObjectCollisions는 적 비행기끼리 충돌했을 때 사용되는 함수이며 과제 1과 동일하다.